

Enterprise Error Handling & Recovery Workflow

Lesson 5 Assessment — Production-Ready n8n Error Handling, Retries & Recovery

This package documents the implemented enterprise error-handling pattern: retry and backoff for transient failures, centralized error handling, PostgreSQL audit logging, Slack operational notification, and validation evidence.

1. Workflow Architecture

Primary Business Workflow: Manual Trigger → HTTP_CallBusinessAPI → Process_SuccessfulResponse.

Centralized Recovery: Error Trigger → PostgreSQL_LogError → Slack_DispatchAlert.

Layer	Implementation
Business	Manual Trigger → HTTP_CallBusinessAPI → Process_SuccessfulResponse
Retry	HTTP Request retry-on-fail; 3 attempts; 5000 ms wait; exponential backoff
Recovery	Error Trigger → PostgreSQL_LogError → Slack_DispatchAlert
Audit	Execution/workflow metadata, failed node, error message, timestamp

2. Retry Strategy & Exponential Backoff

The HTTP request is used as a controlled outbound API failure simulation. Retry On Fail is enabled so transient failures receive automatic recovery attempts before final failure handling.

Setting	Value
Retry On Fail	Enabled
Maximum tries	3
Wait between attempts	5000 ms
Backoff	Exponential
Failure simulation	HTTP endpoint returning a 500-style failure

Expected sequence: Attempt 1 fails → backoff → Attempt 2 fails → backoff → Attempt 3 fails → final failure → Global Error Handler.

3. PostgreSQL Error Logging

The recovery workflow stores one structured failure record for each failed execution. The setup workflow creates the required table and indexes.

```
CREATE TABLE IF NOT EXISTS system_error_logs (  
  id BIGSERIAL PRIMARY KEY,  
  execution_id TEXT NOT NULL,  
  workflow_id TEXT,  
  workflow_name TEXT,  
  failed_node TEXT,  
  error_message TEXT,  
  occurred_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE INDEX IF NOT EXISTS idx_system_error_logs_workflow_id  
ON system_error_logs(workflow_id);  
  
SELECT * FROM system_error_logs  
ORDER BY occurred_at DESC;
```

4. Slack Operational Alert

Slack_DispatchAlert is designed to communicate: Workflow Name, Workflow ID, Execution ID, Failed Node, Error Message, and Execution URL. The node can be enabled with the appropriate Slack credential and operational channel.

5. Node Configuration Blueprint

Node	Purpose
Manual Trigger	Controlled test execution
HTTP_CallBusinessAPI	Outbound API call and failure simulation
Process_SuccessfulResponse	Success-path processing
Error Trigger	Receives failed execution metadata
PostgreSQL_LogError	Persists structured failure audit record
Slack_DispatchAlert	Sends operational failure notification

6. Test Cases & Evidence

Test	Expected result
Successful request	Success processing node runs
Transient failure	HTTP node retries up to configured limit
Final failure	Workflow fails after retries and invokes Error Trigger
Database logging	Structured failure row is inserted
Slack alert	Alert contains required execution metadata
End-to-end recovery	Failure → Error Trigger → PostgreSQL → Slack

The provided execution evidence shows the HTTP failure path and the centralized Error Trigger → PostgreSQL logging → Slack dispatch structure. Add the final retry/database/Slack screenshots to the submission repository.

7. Production Readiness Checklist

- Retry strategy configured
- Exponential backoff applied
- HTTP failure simulated
- Global Error Workflow created
- PostgreSQL logging configured
- Database verification prepared
- Slack operational alert stage configured
- Error workflow linked to primary workflow
- Production naming standards applied
- Test cases defined and executed
- Workflow JSON export prepared
- SQL schema prepared
- Documentation prepared

8. Deployment Instructions

- Import the exported n8n workflow JSON.
- Configure PostgreSQL credentials and create `system_error_logs`.
- Configure Slack credentials and the operational channel.
- Associate the Primary Business Workflow with the Global Error Handler.
- Run a success test, then run the controlled failure test.
- Verify retry attempts, final failure, Error Trigger, PostgreSQL record, and Slack alert.

9. Assumptions, Limitations & Improvements

Testing may use a controlled HTTP 500 endpoint rather than a production API. External credential or service availability can affect live Slack validation. Before production, tune retry behavior for API rate limits and add correlation IDs, alert deduplication, metrics/dashboarding, and dead-letter handling where appropriate.

10. LMS Submission Package

- Loom/YouTube demonstration video
- Public GitHub repository
- Exported n8n workflow JSON

- SQL schema script
- PDF/DOCX documentation
- Test evidence/screenshots
- Deployment instructions
- LMS description/assessor notes